LAPORAN PRATIKUM

PEMBELAJARAN MESIN



NAMA : Maftuh Faisal Jauhari

NIM : 202431163

KELAS : C

DOSEN : Dr. Efy Yosrita, S.Si, M.Kom

NO.PC :

ASISTEN : 1. Muhammad Raka Ilham

2. Muhammad Jabbar

3. Sharira Maharani

4. Muhammmad Fachridzyi

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026

Alhamdullillahiiraabbil alamin saya masih diberi kesehatann sehingga mampu nutuk mengerjakan lapoaran paraktikum 1 dan 2 mata kuliah pembelajaran learning, dengan sadar saya merasa laporan ini penting guna pertimbangan dan evaluasi belajar mahasiswa yang ditinjau oleh segenap asisten.

Praktikum kali ini membahas 5 poroses praktikum yang sudah dipelajari kelima praktikum ini berupa: EDA (Exploratory Data Analysis), Scrapping, Regresi Linear, Decision Tree dan Naïve Bayes. Untuk lebih detailnya say akan breakdown per-praktikum

A. Exploratory Data Analysis (EDA)

Informasi Dataset yang menjadi bahan pengolahan data:

- Judul Dataset: Dirty Manga Sales Data
- Link Kaggle: https://www.kaggle.com/datasets/abhaypanchal/dirty-manga-sales-data
- https://www.kaggle.com/datasets/patelris/anime-and-manga-dataset-2026
- Jumlah Baris: 5000 baris x 24 kolom
- Sifat Data: sintetis (dibuat oleh pembuat dataset, bukan transaksi riil), tapi tetap bermakna untuk latihan EDA karena meniru pola penjualan manga: demografi pelanggan, genre, harga, diskon, rating, platform, dan retur.

Potongan program

```
# Maftuh Faisal_202431163
for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].mean())
    else:
        df[col] = df[col].fillna(df[col].mode()[0])

df.isna().sum()
```

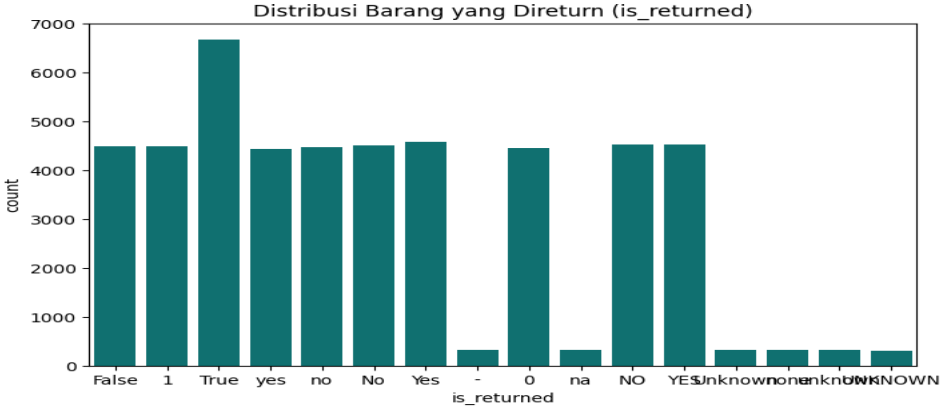
Langkah ini dilakukan untuk mendeteksi dan menangani missing values serta data duplikat pada dataset. Data kosong pada kolom numerik diisi dengan nilai rata-rata (mean) dan kolom kategorik diisi dengan modus, lalu disimpan kembali ke dataframe agar tidak terjadi error saat pemodelan. Setelah itu, baris data yang duplikat dihapus menggunakan drop_duplicates() untuk menjaga validitas data.

```
transaction_id      0
customer_id         0
customer_name       0
age                 0
age_group           0
city                0
email               0
phone               0
manga_title         0
volume_no           0
genre               0
format              0
platform            0
payment_method      0
unit_price          0
quantity            0
total_amount        0
customer_rating     0
purchase_date       0
delivery_days       0
is_returned         0
membership_tier     0
discount_percent    0
coupon_used         0
dtype: int64
```

Visualisasi Data ¶

```
# Maftuh Faisal_202431163
plt.figure(figsize=(8, 5))
sns.countplot(x="is_returned", data=df, color='teal')
plt.title("Distribusi Barang yang Direturn (is_returned)")
plt.show()
```

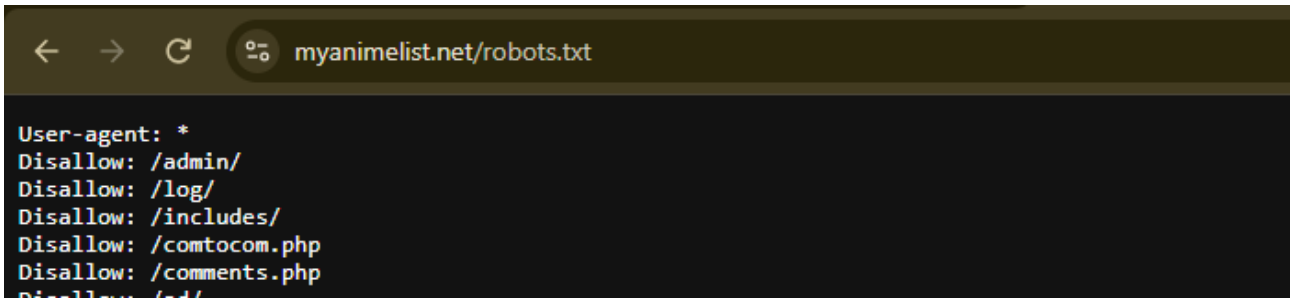
Langkah ini digunakan untuk memvisualisasikan distribusi data pada kolom kategorik is_returned menggunakan pustaka seaborn dan matplotlib. Grafik ini berfungsi untuk melihat perbandingan jumlah transaksi antara barang yang dikembalikan (returned) dan yang tidak dikembalikan secara visual, sehingga kita dapat mengetahui keseimbangan atau sebaran data pada kategori tersebut.



B. Scrapping

Informasi yang perlu diketahui sebelum melakukan scrapping dengan mengecek robots.txt dulu

- Alamat situs robots.txt: https://myanimelist.net/robots.txt
- Isi robots.txt



- Aturan yang berkaitan: Menurut isi file robots.txt dari situs MyAnimeList terdapat aturan yang melarang crawler atau bot mengakses direktori tertentu termasuk bagian pribadi,akun pengguna atau halaman login.Namun halaman direktori utama yang berisi anime dan dimaksudkan untuk di scrap tidak mengandung aturan "disallow" untuk halaman tersebut.
- Kenapa halaman bisa diambil: Pengumpulan data dari halaman ini diijinkan karena tidak melanggar batasan-batasan yang tercantum di robots.txt.Ketersediaan informasi publik tentang anime di situs web ini dapat diakses oleh agen-agen pengguna standar dan juga etis dalam pengumpulan data di web karena tidak ada akses ke informasi pribadi di luar batasan laju situs web tersebut.

Potongan program

Permintaan diajukan melalui protokol HTTP untuk mengakses beberapa halaman di MyAnimeList demi mendapatkan setidaknya 200 data poin tentang anime tersebut. Data HTML dari halaman tersebut kemudian diparse menggunakan BeautifulSoup menggunakan tag baris tabel peringkat (tr) dengan kelas "ranking-list".

```
# Maftuh Faisal_202431163
headers = {
    "User-Agent": (
        "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    )
}

base_url = "https://myanimelist.net/topanime.php?limit={}"
pages_html = []

for limit in [0, 50, 100, 150]:
    url = base_url.format(limit)
    response = requests.get(url, headers=headers)

    print(f"URL: {url} -> status: {response.status_code}")

    if response.status_code != 200:
        raise Exception(f"Gagal mengambil {url}. Status: {response.status_code}")
```

```
URL: https://myanimelist.net/topanime.php?limit=0 -> status: 200
URL: https://myanimelist.net/topanime.php?limit=50 -> status: 200
URL: https://myanimelist.net/topanime.php?limit=100 -> status: 200
URL: https://myanimelist.net/topanime.php?limit=150 -> status: 200

Total halaman berhasil diambil: 4
```

Karena data yang diperoleh melalui web scraping masih dalam bentuk teks (string) kita menggunakan pustaka pandas untuk mengubah nilai-nilai tersebut menjadi variabel numerik (int atau float) untuk variabel seperti episode, skor dan anggota sehingga kita dapat menerapkan model Regresi Linear dan Decision Tree pada dataset tersebut.

```
title           str
type            str
episodes    float64
score       float64
members       int64
dtype: object
Data bersih disimpan ke anime_top_clean.csv
```

```
# Maftuh Faisal_202431163
db["episodes"] = pd.to_numeric(db["episodes"], errors="coerce")
db["score"] = pd.to_numeric(db["score"], errors="coerce")
db["members"] = pd.to_numeric(db["members"], errors="coerce")

print(db.dtypes)

# Simpan hasil bersih
db.to_csv("anime_top_clean.csv", index=False)
print("Data bersih disimpan ke anime_top_clean.csv")
```

C. Regresi Linear

Syarat terpenuhi jadi bisa melakukan perhitungan regresi linear data hasil scrapping praktikum 2

Potongan program

```
Jumlah data setelah dropna: 195
Contoh x:
   episodes  members
0      28.0  1540103
1      64.0  3753044
2      19.0   367248
3      24.0  2864729
4      10.0  2655133

Contoh y:
0    9.24
1    9.11
2    9.07
3    9.07
4    9.06
Name: score, dtype: float64
```

```
# Maftuh Faisal_202431163
db_clean = df.dropna(subset=["episodes", "members", "score"]).copy()

db_clean["episodes"] = pd.to_numeric(db_clean["episodes"], errors="coerce")
db_clean["members"]  = pd.to_numeric(db_clean["members"],  errors="coerce")
db_clean["score"]    = pd.to_numeric(db_clean["score"],    errors="coerce")

db_clean = db_clean.dropna(subset=["episodes", "members", "score"])

x = db_clean[["episodes", "members"]]
y = db_clean["score"]

print("Jumlah data setelah dropna:", len(db_clean))
print("Contoh x:")
print(x.head())
print("\nContoh y:")
print(y.head())
```

Data dibersihkan dari baris yang mengandung nilai kosong (missing values) pada kolom fitur maupun target. Variabel fitur (x) ditentukan dari kolom angka seperti episodes dan members, sedangkan variabel target (y) menggunakan satu atribut angka yaitu score (rating anime), di mana hubungan antar atribut ini masuk akal secara logis.

```
# Maftuh Faisal_202431163
x_train, x_test, y_train, y_test = tts(x, y, test_size=0.2, random_state=0)

print("Jumlah data latih:", len(x_train))
print("Jumlah data uji  :", len(x_test))
```

```
Jumlah data latih: 156
Jumlah data uji  : 39
```

Dataset dibagi menjadi data latih (training set) sebesar 80% dan data uji (testing set) sebesar 20% menggunakan fungsi train_test_split dengan parameter test_size=0.2 dan random_state=0 agar hasil pembagian konsisten.

Model regresi linear dibuat dan dilatih (fit) menggunakan data latih. Selanjutnya, model melakukan prediksi pada data uji dan dievaluasi menggunakan metrik koefisien, intercept, Mean Squared Error (MSE), serta R2 Score untuk mengukur performa prediksi model terhadap data aktual.Berdasarkan nilai koefisien dan intercept yang diperoleh dari hasil pelatihan model, dibentuk persamaan garis regresi multi-variabel ($Y = a + b_1 x1 + b_2 x2$), di mana Y adalah skor anime, x1 adalah jumlah episode, dan x2 adalah jumlah member.

```
# Maftuh Faisal_202431163
model = LinearRegression()
model.fit(x_train, y_train)
```

```
<style>.sk-global { ●●●
```

```
y_pred = model.predict(x_test)
y_pred
```

```
array([8.64282943, 8.62319506, 8.67278984, 8.63891273, 8.76
```

```
print("coefficient :", model.coef_)
print("intercept   :", model.intercept_)
print("MSE         :", mean_squared_error(y_test, y_pred))
print("R2 Score    :", r2_score(y_test, y_pred))
```

```
coefficient : [8.45111151e-04 4.11325841e-08]
intercept   : 8.61538159709783
MSE         : 0.031312641352212
R2 Score    : -0.016587779068983455
```

```
# Maftuh Faisal_202431163
a = model.intercept_
b1, b2 = model.coef_

print("Persamaan regresi:")
print(f"Y = {a:.4f} + {b1:.6f}
print()
print("Dengan:")
print("  Y  = score (rating an
print("  x1 = episodes (jumlah
print("  x2 = members (jumlah
print()
print(f"Interpretasi:")
print(f"  - Setiap tambahan 1
```

```
Persamaan regresi:
Y = 8.6154 + 0.000845·x1 + 0.000000·x2

Dengan:
  Y  = score (rating anime)
  x1 = episodes (jumlah episode)
  x2 = members (jumlah member/popularitas)

Interpretasi:
  - Setiap tambahan 1 episode  → score berubah +0.000845
  - Setiap tambahan 1 member   → score berubah +0.00000004
```

D. Decision Tree

Syarat terpenuhi jadi bisa melakukan perhitungan regresi linear data hasil scrapping praktikum 2

Potongan Program

```
from sklearn.metrics import accuracy_score

y_pred = classifier.predict(x_test)
akurasi = accuracy_score(y_test, y_pred)
print(akurasi)
```

```
0.8166666666666667
```

```
# Maftuh Faisal_202431163
y_pred_new = classifier.predict([[1, 500000, 8.5]])
print(y_pred_new)
print("Prediksi kelas:", le.inverse_transform(y_pred_new))
```

```
[5]
Prediksi kelas: ['TV Special']
```

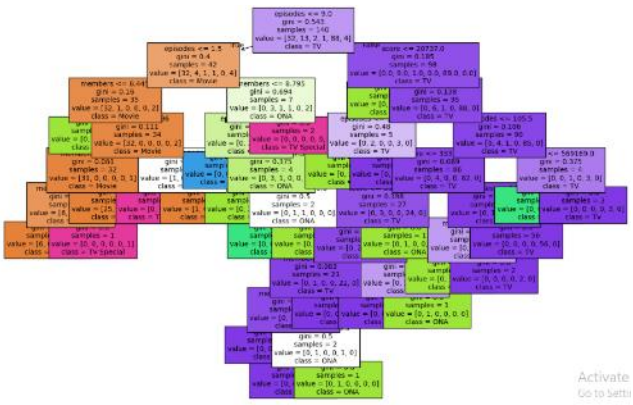
```
from sklearn.metrics import classification_report
semua_label = range(len(le.classes_))
print(classification_report(
    y_test,
    y_pred,
    labels=semua_label,
    target_names=le.classes_,
    zero_division=0
))
```

	precision	recall	f1-score	support
Movie	0.78	0.88	0.82	8
ONA	0.40	0.25	0.31	8
OVA	0.00	0.00	0.00	0
Special	0.00	0.00	0.00	0
TV	0.89	0.93	0.91	43
TV Special	0.00	0.00	0.00	1
accuracy			0.82	60
macro avg	0.34	0.34	0.34	60
weighted avg	0.79	0.82	0.80	60

Logika utama model Decision Tree adalah membangun pohon keputusan dengan cara memilih fitur dan nilai threshold yang paling baik memisahkan kelas pada setiap node. Model ini tidak membutuhkan asumsi distribusi data, sehingga bisa menangkap hubungan non-linear antara episodes, score, dan members terhadap type anime. Target type diubah menjadi angka menggunakan LabelEncoder karena model membutuhkan label numerik. Setelah dilatih dengan 70% data, model diuji dengan 30% data sisa dan menghasilkan akurasi sekitar 81,67%, artinya sekitar 49 dari 60 data testing diprediksi benar. Namun performa tidak merata: kelas TV dan Movie diprediksi cukup baik (F1 masing-masing 0,91 dan 0,82), sedangkan ONA, OVA, Special, dan TV Special buruk karena jumlah sampel testing sangat sedikit atau bahkan nol. Nilai macro avg yang rendah (0,34) menandakan model tidak adil untuk kelas minoritas. Saat diberi input baru [[1, 500000, 8.5]], model memprediksi kelas TV Special, yang menunjukkan model mengandalkan kombinasi episode, jumlah member, dan score untuk menentukan kelas.

```
from sklearn import tree
# Maftuh Faisal_202431163
plt.subplots(figsize=(15, 10))
tree.plot_tree(
    classifier,
    feature_names=['episodes', 'members', 'score'],
    class_names=le.classes_.tolist(),
    filled=True,
    fontsize=10
)
plt.show()
```

jika kita membaca gambar pohonnya dari atas ke bawah: Kotak paling atas adalah akar (root node). Model melihat bahwa fitur paling penting untuk



membedakan tipe anime adalah jumlah episode (episodes <= 9.0). Jika jumlah episode lebih dari 9, data akan masuk ke cabang kanan (dominan berwarna ungu, kelas TV). Jika kurang dari sama dengan 9, data akan masuk ke cabang kiri (dominan berwarna oranye, kelas Movie).Proses ini berulang terus ke bawah menggunakan fitur members dan score, sampai data benar-benar terpisah. Kotak paling bawah disebut daun (leaf node) yang punya gini = 0.0 dan samples = 1, artinya pohon sudah berhasil mengelompokkan data hingga murni 100% satu kelas saja.

E. Naïve Bayes

Informasi Dataset yang menjadi bahan pengolahan data:

- Judul Dataset: Anime and manga Analytics Dataset
- Link Kaggle: https://www.kaggle.com/datasets/patelris/anime-and-manga-dataset-2026
- Jumlah Baris: 828889 baris x 24 kolom

Potongan Program

```
# Maftuh Faisal_202431163
from sklearn.metrics import confusion_matrix, classification_report

print("Evaluasi Testing")
cm_test = confusion_matrix(y_test, y_pred_test)
print(cm_test)
print(classification_report(y_test, y_pred_test))

plt.figure(figsize = (6, 4))
sns.heatmap(cm_test, annot = True, fmt='d', cmap = 'Greens', xticklabels = np.unique(y), yticklabels = np.unique(y))
plt.title("Confusion Matrix - Testing")
plt.xlabel("Label Terprediksi")
plt.ylabel("Label Asli")
plt.show()
```

```
Evaluasi Testing
[[    7    38     5     8]
 [  705 12081   330  4048]
 [   10    72     6    23]
 [    0     1    14  3375]]
              precision    recall  f1-score   support

           0       0.01      0.12      0.02        58
           1       0.99      0.70      0.82     17164
           2       0.02      0.05      0.03       111
           3       0.45      1.00      0.62      3390

    accuracy                           0.75     20723
   macro avg       0.37      0.47      0.37     20723
weighted avg       0.89      0.75      0.78     20723
```

```
new_data = {}
for col in x.columns:
    if col in numerical_features:
        new_data[col] = [data[col].median()]
    else:
        new_data[col] = [data[col].mode()[0]]

new_df = pd.DataFrame(new_data)
prediction = model.predict(new_df)
print("Hasil Prediksi Data Baru:", prediction[0])
```

```
Hasil Prediksi Data Baru: 3
```

Logika utama Gaussian Naive Bayes adalah menghitung probabilitas setiap kelas berdasarkan asumsi bahwa tiap fitur numerik berdistribusi normal (Gaussian) di dalam masing-masing kelas. Model menghitung mean dan variance setiap fitur per kelas selama training, lalu saat prediksi memakai teorema Bayes untuk memilih kelas dengan probabilitas tertinggi. Karena GaussianNB hanya menerima input numerik, seluruh fitur kategorikal seperti type, authors, genres, dan themes harus diubah dulu dengan LabelEncoder. Model dilatih dengan 75% data dan diuji dengan 25% data, menghasilkan akurasi testing sekitar 75%. Namun angka ini menyesatkan karena data sangat tidak seimbang: kelas 1 (Finished) mendominasi dengan jumlah 17.164 sampel testing, sedangkan kelas 0 hanya 58 dan kelas 2 hanya 111. Akibatnya model bias ke kelas mayoritas, terlihat dari precision kelas 1 sebesar 0,99 tetapi recall hanya 0,70. Kelas minoritas 0 dan 2 hampir tidak terprediksi sama sekali (F1 0,02 dan 0,03), sedangkan kelas 3 (Publishing) punya recall 1,00 tetapi precision 0,45 karena banyak kelas lain yang salah dimasukkan ke kelas 3. Nilai macro avg hanya 0,37, menunjukkan model tidak adil untuk kelas minoritas. Saat diberi input baru berupa nilai median semua fitur, model memprediksi kelas 3 (Publishing), tetapi prediksi ini lemah karena model sangat dipengaruhi dominasi kelas mayoritas selama training.

Dengan ini telah selesai laporan praktikum mesin belajar 1 dan 2 untuk selengkapnya saya taruh program di github yang tertera di pengumpulan assigment ms teams.